Lab 02: GitHub API

Stalking z klasą

Jarosław Hryszko
jaroslaw.hryszko@uj.edu.pl

Instytut Informatyki i Matematyki Komputerowej UJ

Otwarte repozytoria kodu i pomiary oprogramowania

Poprzednio w Git Archaeology...

- Kopaliśmy w historii gita **lokalnie** (`git log`, `git blame`)
- Dziś idziemy dalej – sięgamy po dane z **serwera GitHub**
- Issues, pull requesty, statystyki kontrybutorów, aktywność
- Wszystko dostępne programowo przez **REST API**

GitHub REST API – podstawy

Scenariusz

Szef mówi: znajdź nam bibliotekę do X, ale taką, która jest aktywnie rozwijana. Wchodzisz na GitHub – ale ręczne klikanie po 50 repozytoriach to koszmar. API pozwala odpytać setki repo w minuty.

Jak to działa?

- Zapytania HTTP GET do `https://api.github.com/...`
- Odpowiedzi w formacie JSON
- Autentykacja: Personal Access Token (PAT)
- Rate limit: 5000 req/h z tokenem, 60 bez

Pułapki, o których warto wiedzieć

1. **Paginacja** – API domyślnie zwraca 30 elementów (max 100). Nagłówek `Link` w odpowiedzi zawiera URL następnej strony – trzeba go parsować i iterować, aż `rel="next"` zniknie.
2. **Rate limiting** – 5000/h brzmi dużo, ale zapytanie per issue per komentarz szybko zjada limit.
3. **Issues vs PRs** – endpoint `/issues` zwraca **też pull requesty**! Jeśli obiekt JSON ma klucz `"pull_request"` (z URLami) – to PR, nie issue.
4. **Daty** – format ISO 8601 z "Z" na końcu.

Wskazówki: parsowanie dat z API

Daty z GitHub API wyglądają tak: "2024-03-15T10:30:00Z"

```
from datetime import datetime

# Python 3.11+:
dt = datetime.fromisoformat("2024-03-15T10:30:00Z")

# Python 3.9-3.10:
raw = "2024-03-15T10:30:00Z"
dt = datetime.fromisoformat(
    raw.replace("Z", "+00:00")
)

# Roznica w dniach miedzy dwiema datami:
delta = (dt2 - dt1).total_seconds() / 86400
```

Wskazówki: analiza issues

Filtrowanie – issues vs pull requesty

Endpoint `/repos/{owner}/{repo}/issues` zwraca **oba**. Prawdziwe issue nie ma klucza `"pull_request"`:

```
issues = [i for i in all_items
          if "pull_request" not in i]
```

Czas do pierwszej odpowiedzi

- Pole `issue["comments"]` mówi ile jest komentarzy
- URL do komentarzy: `issue["comments_url"]`
- Pierwszy komentarz = `?per_page=1` – oszczędność zapytań
- Ogranicz się do np. 20 issues, żeby nie zjeść rate limitu

Wskazówki: analiza pull requestów

Merged vs odrzucone vs otwarte

Endpoint: `/repos/{owner}/{repo}/pulls?state=all`

Każdy PR ma pola:

- `"state"`: `"open"` lub `"closed"`
- `"merged_at"`: data merge'a lub `null`

```
merged = [p for p in prs if p.get("merged_at")]
rejected = [p for p in prs
            if p["state"] == "closed"
            and not p.get("merged_at")]
still_open = [p for p in prs
              if p["state"] == "open"]
```

Wskazówki: analiza aktywności

Committy w zadanym okresie

Endpoint `/repos/{owner}/{repo}/commits` przyjmuje parametry:

- `since` – data ISO 8601 (od kiedy)
- `until` – data ISO 8601 (do kiedy)

```
from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc)
month_ago = now - timedelta(days=30)

commits = fetch_paginated(url, headers,
    {"since": month_ago.isoformat()},
    max_items=100)
print(f"Commitow: {len(commits)}")
```

Porownanie z rokiem temu: ten sam trik, ale `since` = 365 dni temu, `until` = 335 dni temu.

Wskazówki: etykiety issues

Top etykiety

Każdy issue ma pole "labels" – listę obiektów z kluczem "name".

```
from collections import Counter

label_counter = Counter()
for issue in issues:
    for label in issue.get("labels", []):
        label_counter[label["name"]] += 1

top5 = label_counter.most_common(5)
```

Plan zajęć

1. **Zadanie 1 – Rozgrzewka z API** (30 min)
   - Token, curl, pierwsze zapytania w Pythonie
   - Sprawdzenie rate limitu
2. **Zadanie 2 – GitHub Profiler** (60 min)
   - Skrypt `github_profiler.py`
   - Profil repo: metryki, issues, PRs, aktywność
3. **Zadanie 3 – Porównywarka** (45 min, opcjonalne)
   - Dwa repo obok siebie: które jest zdrowsze?

Co oddajecie?

W branchu `lab02_nazwisko1_nazwisko2`:

1. `github_profiler.py` – działający skrypt
2. `report.txt` – output dla wybranego repo
3. *(opcjonalnie)* `compare_repos.py`

Uwaga

Nie commitujcie tokena! Używajcie `os.environ["GITHUB_TOKEN"]`.

Do roboty!

Instrukcja: `README.md` w repozytorium

"*Jeśli nie możesz tego zmierzyć, nie możesz tym zarządzać.*" – Peter Drucker (albo ktoś inny)